

Instruções

1. Esta avaliação deve ser feita em trio.
2. Data de entrega: **21/09/2026** até as 23:59. Não serão aceitos trabalhos em atraso.
3. Esta avaliação tem por objetivo consolidar o aprendizado sobre conceitos de IPC, threads, concorrência e paralelismo.
4. A implementação deverá ser desenvolvida utilizando a linguagem de programação C/C++. Porém, a utilização e suporte a IPC e threads pela linguagem é de responsabilidade do(s) aluno(s). Deve ser usado corretamente o conceito de threads e IPC. As bibliotecas usadas devem ser equivalentes a Pthreads. Bibliotecas que também implementem e que permitam usar conceitos de paralelismo também podem ser usadas, mas o aluno também é responsável pelo seu uso e apresentação.
5. O sistema deve ser entregue funcionando corretamente. Sistemas não compilando e executando poderão não ser aceitos. É de responsabilidade do(s) aluno(s) apresentar a execução funcionando corretamente bem como o que será enviado.
6. Deve ser disponibilizado os códigos da implementação em repositório do(s) aluno(s) (github, bitbucket, etc...), deve ser fornecido o link e o repositório deve ser público.
7. O relatório sobre a atividade deve seguir o formato PDF de artigo científico ou seguindo as normas da ABNT e às orientações para produção de trabalhos acadêmicos da Univali contendo:
 - Identificação do autor e do trabalho.
 - Enunciado dos projetos.
 - Explicação e contexto da aplicação para compreensão do problema tratado pela solução.
 - Resultados obtidos com as simulações.
 - Códigos importantes de implementação.
 - Resultados obtidos com a implementação (tabelas, gráficos e etc).
 - Análise e discussão sobre os resultados finais.
8. Deve ser disponibilizado os códigos da implementação juntamente com o relatório (salvo o caso da disponibilidade em repositório aberto do aluno, que deve ser fornecido o link). O repositório deve estar aberto do momento da entrega em diante, sendo que o professor não se responsabiliza caso o projeto não esteja disponível para consulta no momento da correção, sendo do(s) aluno(s) essa responsabilidade de manter disponível. Cópias entre alunos implicará em nota zero para todos os envolvidos.
9. O trabalho deverá ser apresentado em data definida pelo professor. É de responsabilidade do(s) aluno(s) explicar os conceitos, comandos, bibliotecas usadas. É de responsabilidade do(s) aluno(s) fazer a solução funcionar e ela deverá ser baixada do local de entrega no momento da apresentação. Os trabalhos não apresentados terão como nota máxima 4,0, além dos descontos aplicados no restante da avaliação da implementação.
10. Todos os alunos poderão apresentar ou o professor poderá escolher um representante para apresentar o trabalho em nome do grupo.

Descrição do projeto a ser desenvolvido

Problemática: Sistema de Processamento Paralelo de Requisições a um Banco de Dados

Descrição Geral

Você irá desenvolver um sistema que simula o funcionamento interno de um **gerenciador de requisições a um banco de dados**, com múltiplos processos e threads:

- Um **processo cliente** envia requisições de consulta ou inserção via IPC (pipe ou memória compartilhada);
- Um **processo servidor** recebe as requisições e usa várias **threads** para processá-las em paralelo;
- As threads utilizam **mutex ou semáforo** para garantir acesso seguro a uma estrutura compartilhada que simula um banco de dados (como um vetor ou um arquivo).

Componentes do Sistema

1. Processo Cliente

- Envia "requisições de banco" para o servidor:
 - Exemplo de requisição: `SELECT nome WHERE id=5` ou `INSERT id=7 nome='João'`
- A comunicação é feita via **pipe** ou **memória compartilhada**.

2. Processo Servidor (Gerenciador de Banco)

- Lê as requisições do pipe ou memória compartilhada.
- Cria um **pool de threads** para processar cada requisição.
- As threads acessam um "banco de dados simulado" (pode ser um vetor ou arquivo .txt/.json/.sqlite) e usam **mutex/semáforo** para garantir a integridade dos dados.
- As respostas podem ser salvas em um segundo canal IPC, ou em um arquivo de log.

Requisições Suportadas

- **INSERT**: Adiciona um novo registro (linha) no "banco".
- **DELETE**: Remove um registro.
- **SELECT**: Busca e retorna dados com base em critérios simples do processos (por ID).
- **UPDATE**: Modifica um campo de um registro (por ID).

Exemplo de estrutura interna do "banco":

```
typedef struct {  
    int id;  
    char nome[50];  
} Registro;
```

A exclusão mútua é um mecanismo fundamental em computação concorrente que garante que apenas um processo ou thread acesse um recurso compartilhado (seção crítica) por vez.

Requisitos técnicos obrigatórios:

- Cliente e servidor são **dois binários/executáveis distintos**, rodando como **processos de SO separados** (não um único processo com duas threads simulando "cliente" e "servidor").
- Comunicação entre eles usa **IPC real** (pipe nomeado/FIFO, memória compartilhada POSIX.) — não variáveis globais, arquivos temporários lidos por polling ingênuo, nem chamadas de função direta.
- O servidor usa **múltiplas threads** (pool) para atender requisições em paralelo.
- O acesso à tabela compartilhada entre as threads é protegido por **mutex ou semáforo real** (pthread_mutex_t, sem_t).

Uso de Threads e Mutex/Semáforo

- Threads compartilham a mesma "tabela de dados".
- Acesso à tabela (inserção, delete) é protegido por:
 - pthread_mutex_t (para exclusão mútua), ou
 - sem_t (para controle mais geral, como múltiplas leituras simultâneas).
- Isso evita condições de corrida e garante a consistência dos dados.
- Adapte no desenvolvimento do trabalho.

